Solution: Contiguous Array

Problem Statement

Given an array `nums` containing only `1` and `0` digits, return the `maximum length` of the `contiguous subarray` having an equal number of `1` and `0`.

Examples

- **Example 1**:
  - **Input**: `[0, 1]`
  - **Expected Output**: `2`
  - **Justification**: The longest subarray with equal numbers of 0s and 1s is [0, 1].
- **Example 2**:
  - **Input**: `[1, 1, 0, 0, 1, 1, 0]`
  - **Expected Output**: `6`
  - **Justification**: The subarray [1, 0, 0, 1, 1, 0] contains three 0s and three 1s.
- **Example 3**:
  - **Input**: [0, 1, 1, 0, 1, 0, 0, 1, 1]`
  - **Expected Output**: `8`
  - **Justification**: The subarray from the first to the eighth element is the longest with an equal number of 0s and 1s.

Solution

To solve this problem, we'll utilize a hash map to keep track of the balance between 0s and 1s at each index. The balance is calculated by treating 0 as -1 and 1 as +1 and adding these values cumulatively. The hash map will store each unique cumulative balance with its earliest index. If we encounter the same balance again, it means the subarray between these two indices has an equal number of 0s and 1s. The reason this approach is effective is that it reduces the problem to a single pass through the array, updating the hash map and checking for the longest balanced subarray as we go.

Step-by-step Algorithm

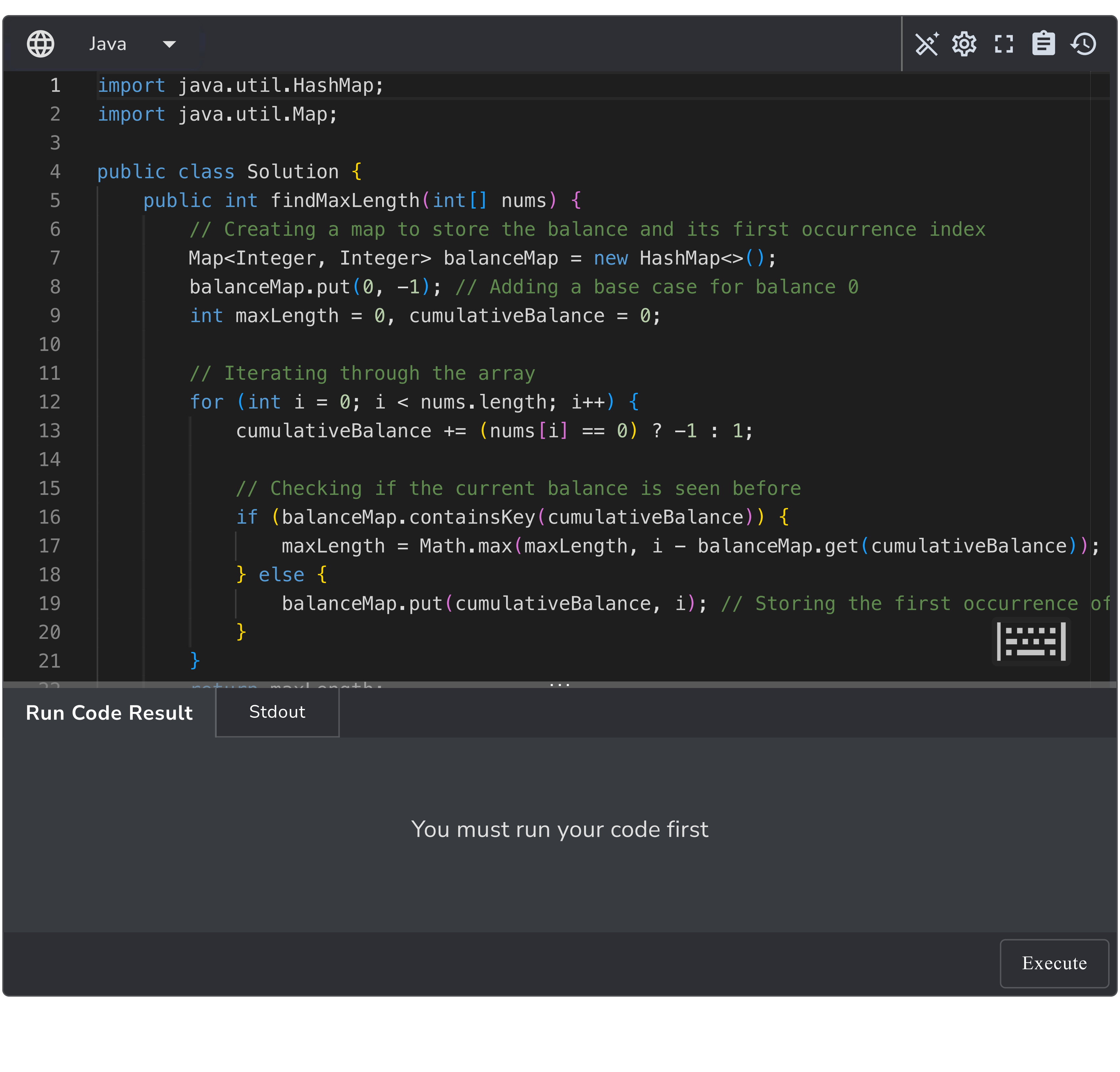
1. **Initialize Variables**:
   - Create a map/dictionary `balanceMap` to keep track of cumulative balances and their first occurrence indexes.
   - Initialize two variables: `cumulativeBalance` to 0 (to keep track of the balance between 0s and 1s) and `maxLength` to 0 (to store the length of the longest subarray found).
2. **Iterate Through the Array**:
   - Loop through each element of the array, using a variable `i` for indexing.
3. **Update Cumulative Balance**:
   - For each element, update `cumulativeBalance`. Decrease by 1 if the element is 0; otherwise, increase by 1.
4. **Check and Update the Map**:
   - If `cumulativeBalance` is already present in `balanceMap`, calculate the length of the current balanced subarray by subtracting the index stored in `balanceMap[cumulativeBalance]` from the current index `i`. Update `maxLength` if this length is greater than the current `maxLength`.
   - If `cumulativeBalance` is not present in `balanceMap`, add it to the map with the value of the current index `i`.
5. **Return the Result**:
   - After the loop, return `maxLength` as the result.

Algorithm Walkthrough

Using Example 2: `[1, 1, 0, 0, 1, 1, 0]`

1. **Initialization**:
   - `balanceMap = {0: -1}`
   - `cumulativeBalance = 0`, `maxLength = 0`
2. **Iterate and Update**:
   - **Index 0** (Element 1):
     - Update `cumulativeBalance` to 1.
     - Update `balanceMap`: `{0: -1, 1: 0}`
     - `maxLength`: 0 (no change).
   - **Index 1** (Element 1):
     - Update `cumulativeBalance` to 2.
     - Update `balanceMap`: `{0: -1, 1: 0, 2: 1}`
     - `maxLength`: 0 (no change).
   - **Index 2** (Element 0):
     - Update `cumulativeBalance` to 1 (since it's seen before, check for subarray).
     - `balanceMap`: `{0: -1, 1: 0, 2: 1}`
     - `maxLength`: `max(0, 2 - 0) = 2` (subarray from index 0 to 2).
   - **Index 3** (Element 0):
     - Update `cumulativeBalance` to 0 (seen before, check for subarray).
     - `balanceMap`: `{0: -1, 1: 2, 2: 1}`
     - `maxLength`: `max(2, 3 - (-1)) = 4` (subarray from index 0 to 3).
   - **Index 4** (Element 1):
     - Update `cumulativeBalance` to 1 (seen before, check for subarray).
     - `balanceMap`: `{0: -1, 1: 0, 2: 1}`
     - `maxLength`: `max(4, 4 - 0) = 4` (no change).
   - **Index 5** (Element 1):
     - Update `cumulativeBalance` to 2 (seen before, check for subarray).
     - `balanceMap`: `{0: -1, 1: 0, 2: 1}`
     - `maxLength`: `max(4, 5 - 1) = 4` (no change).
   - **Index 6** (Element 0):
     - Update `cumulativeBalance` to 1 (seen before, check for subarray).
     - `balanceMap`: `{0: -1, 1: 0, 2: 1}`
     - `maxLength`: `max(4, 6 - 0) = 6` (subarray from index 0 to 6).
3. **Result**:
   - Final `maxLength` is 6, which is the length of the longest contiguous subarray with an equal number of 0s and 1s.

Code



```java
import java.util.HashMap;
import java.util.Map;

public class Solution {
    public int findMaxLength(int[] nums) {
        // Creating a map to store the balance and its first occurrence index
        Map<Integer, Integer> balanceMap = new HashMap<>();
        balanceMap.put(0, -1); // Adding a base case for balance 0
        int maxLength = 0, cumulativeBalance = 0;

        // Iterating through the array
        for (int i = 0; i < nums.length; i++) {
            cumulativeBalance += (nums[i] == 0) ? -1 : 1;

            // Checking if the current balance is seen before
            if (balanceMap.containsKey(cumulativeBalance)) {
                maxLength = Math.max(maxLength, i - balanceMap.get(cumulativeBalance));
            } else {
                balanceMap.put(cumulativeBalance, i); // Storing the first occurrence of
            }
        }
```

Complexity Analysis

Time Complexity

The algorithm traverses the array once so time complexity is `O(n)`, where `n` is the length of the array. Map/dictionary operations like insertion and lookup take `O(1)` time on average, so the time complexity is linear.

Space Complexity

In the worst case, the map/dictionary might store an entry for every element in the array, leading to a space complexity proportional to the size of the input array, i.e., `O(n)` .

← Previous
Contiguous Array (Medium)
Next →
Sliding Window Median (Hard)
☐ Mark as Completed